جامعة الملك سعود
King Saud University
كلية علوم الحاسب والمعلومات
قسم نظم المعلومات

King Saud University
College of Computer and Information Sciences
Department of Information Systems

IS230: Introduction to Database Systems
2nd Semester 2025

Transitly

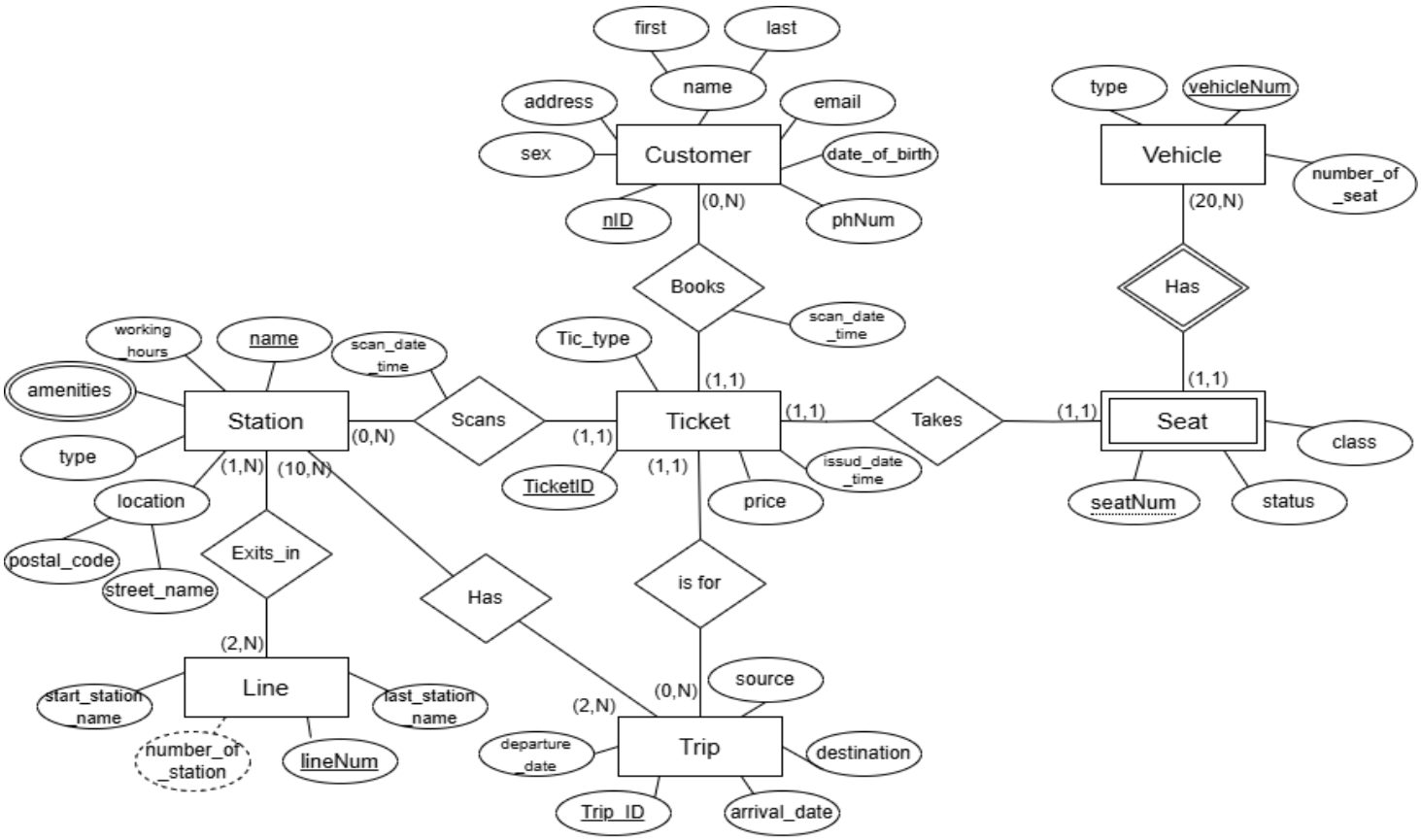
Phase # 1

Section #	SN	NAME	ID
Group Number: 5			
67166	*#*	*Madhawi sanad alsanad*	*445202205*
67166	*#*	*Lubna Sami Alrashoud*	*445204219*
67166	*#*	*Ashwaq Ahmed Almuhaysin*	*444200737*
67166	*#*	*Alanoud naif Alotaibi*	*445202172*
67166	*#*	*layan sultan bin saleh*	*445202108*

Supervised By: *Dr. Lubna Yousef Alkhalil.*

Part 1:

a) Entity relationship diagram (ER):



Note:
- Ticket is for one Trip, and a Trip can have multiple Tickets.
Assumption: Each ticket must be linked to a specific trip so we can know which ride the customer is taking.

- Trip has 2 or multiple Stations.
Assumption: A trip must have multiple stations because it starts at one station and ends at another, passing through several stations along the way.

- Line has two or more Stations.
Assumption: A line must have at least two stations to complete a route, as it needs to connect different stations, allowing passengers to travel between them.

جامعة الملك سعود
King Saud University
كلية علوم الحاسب والمعلومات
قسم نظم المعلومات

King Saud University
College of Computer and Information Sciences
Department of Information Systems

IS230: Introduction to Database Systems
2nd Semester

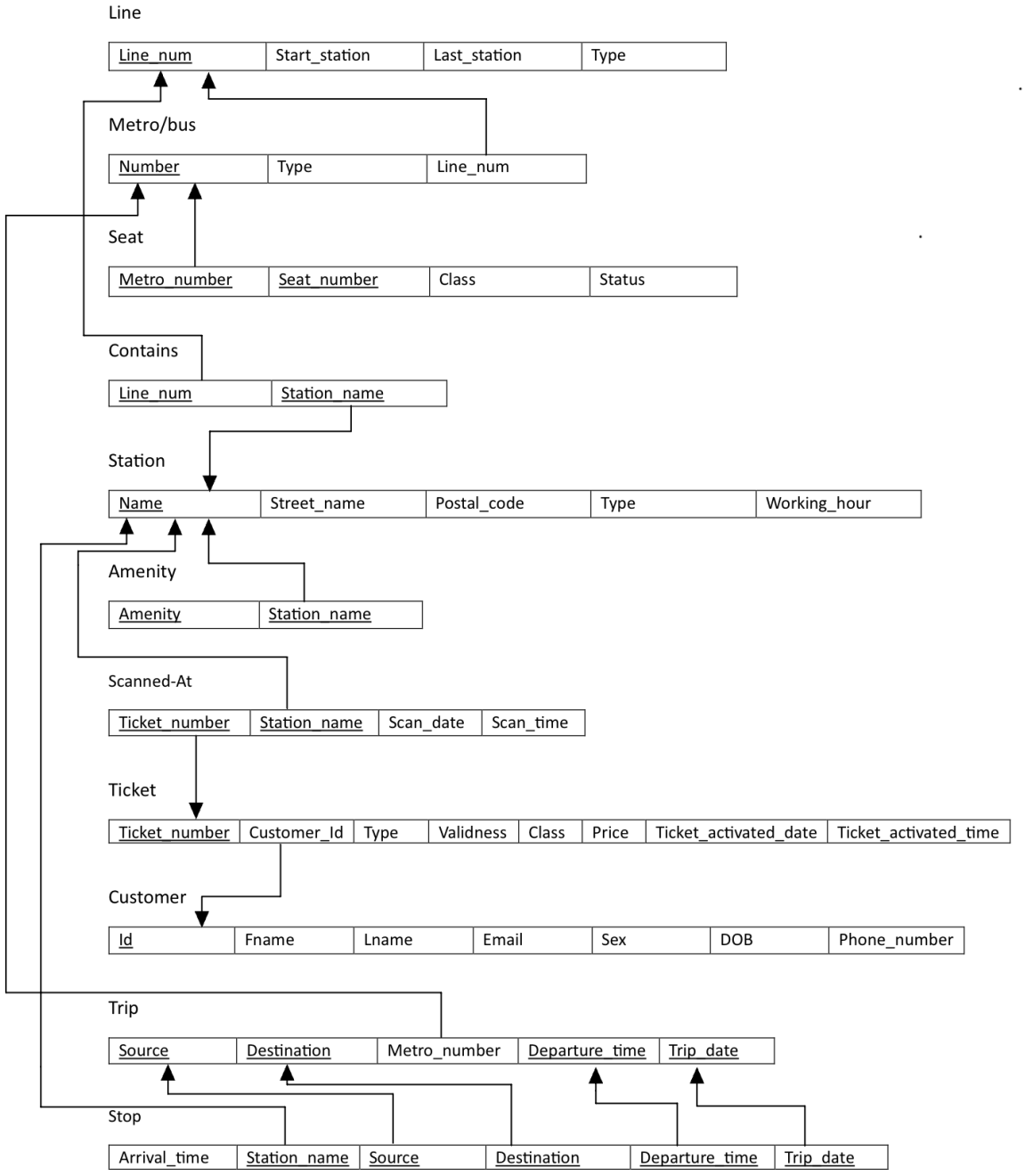
Transitly
Phase # 2

Group #: *5*

Section #	NAME	ID
67166	*Madhawi sanad Alsanad*	*445202205*
67166	*Ashwaq Ahmed Almuhaysin*	*444200737*
67166	*Alanoud Naif Alotaibi*	*445202172*
67166	*Lubna Sami Alrashoud*	*445204219*
67166	*layan sultan bin saleh*	*445202108*

Part 1:

a) Database schema:



b) SQL script to generate the Database:

```
CREATE SCHEMA Project;
USE project;
CREATE TABLE line(
   line_num INT PRIMARY KEY,
   start_station VARCHAR(30),
   last_station VARCHAR(30),
   type VARCHAR(5) CHECK (type IN('metro', 'bus'))
   );
CREATE TABLE Metro_Bus(
   number INT PRIMARY KEY,
   type VARCHAR(5) CHECK (type IN('metro', 'bus')),
   line_num INT,
   FOREIGN KEY(line_num) REFERENCES line(line_num)
   );
CREATE TABLE Seat(
   metro_number INT,
   seat_number VARCHAR(3),
   class VARCHAR(6) CHECK (class IN('first', 'normal')),
   status VARCHAR(10) CHECK (status IN('available', 'booked')),
   PRIMARY KEY(metro_number, seat_number),
   FOREIGN KEY(metro_number) REFERENCES Metro_Bus(number) ON DELETE CASCADE
   );
CREATE TABLE Station(
   name VARCHAR(30) PRIMARY KEY,
   street_name VARCHAR(30),
   postal_code CHAR(5),
   type VARCHAR(5) CHECK (type IN('metro', 'bus')),
   working_hours VARCHAR(15)
   );

CREATE TABLE Contains(
   line_num INT,
   station_name VARCHAR(30),
   PRIMARY KEY(line_num, station_name),
   FOREIGN KEY(line_num) REFERENCES line(line_num) ON DELETE CASCADE,
   FOREIGN KEY(station_name) REFERENCES station(name) ON DELETE CASCADE
   );
CREATE TABLE Amenity(
   amenity VARCHAR(30),
   station_name VARCHAR(30),
   PRIMARY KEY(amenity, station_name),
   FOREIGN KEY(station_name) REFERENCES station(name) ON DELETE CASCADE
   );
```

```
CREATE TABLE Customer (
Id CHAR(10) PRIMARY KEY,
Fname VARCHAR(20),
Lname VARCHAR(20),
Email VARCHAR(35),
Sex VARCHAR(6) CHECK (sex IN ('Male', 'Female')),
DOB DATE,
Phone_number VARCHAR(15));

CREATE TABLE Ticket (
Ticket_number INT PRIMARY KEY,
Customer_Id VARCHAR(10),
Type VARCHAR(8) CHECK (type IN ('2 hours', '3 days', '7 days', '30 days')),
Validness VARCHAR(10)  CHECK (validness IN ('valid', 'invalid')),
Class VARCHAR(10) CHECK (class IN ('First', 'Normal')),
Price DECIMAL(6,2),
Ticket_activated_date DATE,
Ticket_activated_time TIME,
FOREIGN KEY (Customer_Id) REFERENCES Customer(Id));

CREATE TABLE Scanned_At (
Ticket_number INT,
Station_name VARCHAR(30),
Scan_date DATE,
Scan_time TIME,
PRIMARY KEY (Ticket_number, Station_name),
FOREIGN KEY (Ticket_number) REFERENCES Ticket(Ticket_number),
FOREIGN KEY (Station_name) REFERENCES Station(Name));

CREATE TABLE Trip (
Source VARCHAR(30),
Destination VARCHAR(30),
Metro_number INT,
Departure_time TIME,
Trip_date DATE,
PRIMARY KEY (Source, Destination, Departure_time, Trip_date),
FOREIGN KEY (Metro_number) REFERENCES Metro_Bus(Number));

CREATE TABLE Stop (
Arrival_time TIME,
Station_name VARCHAR(30),
Source VARCHAR(30),
Destination VARCHAR(30),
Departure_time TIME,
Trip_date DATE,
PRIMARY KEY (Station_name, Source, Destination, Departure_time, Trip_date),
FOREIGN KEY (Station_name) REFERENCES Station(Name),
FOREIGN KEY (Source, Destination, Departure_time, Trip_date) REFERENCES Trip(Source, Destination,
Departure_time, Trip_date));
```

c) Screenshot of applying the SQL script in MariaDB:

```
MariaDB [(none)]> CREATE SCHEMA Project;
Query OK, 1 row affected (0.002 sec)
```

```
MariaDB [(none)]> USE project;
Database changed
MariaDB [project]>
```

```
MariaDB [project]> CREATE TABLE line(
    -> line_num INT PRIMARY KEY,
    -> start_station VARCHAR(30),
    -> last_station VARCHAR(30),
    -> type VARCHAR(5) CHECK (type IN('metro', 'bus'))
    -> );
Query OK, 0 rows affected (0.015 sec)
```

```
MariaDB [project]> CREATE TABLE Metro_Bus(
    ->     number INT PRIMARY KEY,
    ->     type VARCHAR(5) CHECK (type IN('metro', 'bus')),
    ->     line_num INT,
    ->     FOREIGN KEY(line_num) REFERENCES line(line_num)
    ->     );
Query OK, 0 rows affected (0.021 sec)
```

```
MariaDB [project]> CREATE TABLE Seat(
    -> metro_number INT,
    -> seat_number VARCHAR(3),
    -> class VARCHAR(6) CHECK (class IN('first', 'normal')),
    -> status VARCHAR(10) CHECK (status IN('available', 'booked')),
    -> PRIMARY KEY(metro_number, seat_number),
    -> FOREIGN KEY(metro_number) REFERENCES Metro_Bus(number) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.013 sec)
```

```
MariaDB [project]> CREATE TABLE Station(
    -> name VARCHAR(30) PRIMARY KEY,
    -> street_name VARCHAR(30),
    -> postal_code CHAR(5),
    -> type VARCHAR(5) CHECK (type IN('metro', 'bus')),
    -> working_hours VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.014 sec)
```

```
MariaDB [project]> CREATE TABLE Contains(
    -> line_num INT,
    -> station_name VARCHAR(30),
    -> PRIMARY KEY(line_num, station_name),
    -> FOREIGN KEY(line_num) REFERENCES line(line_num) ON DELETE CASCADE,
    -> FOREIGN KEY(station_name) REFERENCES station(name) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.017 sec)
```

```
MariaDB [project]> CREATE TABLE Amenity(
    -> amenity VARCHAR(30),
    -> station_name VARCHAR(30),
    -> PRIMARY KEY(amenity, station_name),
    -> FOREIGN KEY(station_name) REFERENCES station(name) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.014 sec)
```

```
MariaDB [project]> CREATE TABLE Customer (
    -> Id CHAR(10) PRIMARY KEY,
    -> Fname VARCHAR(20),
    -> Lname VARCHAR(20),
    -> Email VARCHAR(35),
    -> Sex VARCHAR(6) CHECK (sex IN ('Male', 'Female')),
    -> DOB DATE,
    -> Phone_number VARCHAR(15));
Query OK, 0 rows affected (0.014 sec)
```

```
MariaDB [project]> CREATE TABLE Ticket (
    -> Ticket_number INT PRIMARY KEY,
    -> Customer_Id VARCHAR(10),
    -> Type VARCHAR(8) CHECK (type IN ('2 hours', '3 days', '7 days', '30 days')),
    -> Validness VARCHAR(10)  CHECK (validness IN ('valid', 'invalid')),
    -> Class VARCHAR(10) CHECK (class IN ('First', 'Normal')),
    -> Price DECIMAL(6,2),
    -> Ticket_activated_date DATE,
    -> Ticket_activated_time TIME,
    -> FOREIGN KEY (Customer_Id) REFERENCES Customer(Id));
Query OK, 0 rows affected (0.015 sec)
```

```
MariaDB [project]> CREATE TABLE Scanned_At (
    -> Ticket_number INT,
    -> Station_name VARCHAR(30),
    -> Scan_date DATE,
    -> Scan_time TIME,
    -> PRIMARY KEY (Ticket_number, Station_name),
    -> FOREIGN KEY (Ticket_number) REFERENCES Ticket(Ticket_number),
    -> FOREIGN KEY (Station_name) REFERENCES Station(Name));
Query OK, 0 rows affected (0.016 sec)
```

```
MariaDB [project]> CREATE TABLE Trip (
    -> Source VARCHAR(30),
    -> Destination VARCHAR(30),
    -> Metro_number INT,
    -> Departure_time TIME,
    -> Trip_date DATE,
    -> PRIMARY KEY (Source, Destination, Departure_time, Trip_date),
    -> FOREIGN KEY (Metro_number) REFERENCES Metro_Bus(Number));
Query OK, 0 rows affected (0.016 sec)
```

```
MariaDB [project]> CREATE TABLE Stop (
   -> Arrival_time TIME,
   -> Station_name VARCHAR(30),
   -> Source VARCHAR(30),
   -> Destination VARCHAR(30),
   -> Departure_time TIME,
   -> Trip_date DATE,
   -> PRIMARY KEY (Station_name, Source, Destination, Departure_time, Trip_date),
   -> FOREIGN KEY (Station_name) REFERENCES Station(Name),
   -> FOREIGN KEY (Source, Destination, Departure_time, Trip_date) REFERENCES Trip(Source, Destination, Departure_time, Trip_date));
Query OK, 0 rows affected (0.017 sec)
```

Part 2:

Screenshot of the execution showing how clear and specific messages will be displayed for each.

1) First Screen:

```
==== Station Table Menu ====
1. Insert new record.
2. Display all records.
3. Exit.
Choose an operation (^_^):
```

2) INSERT Operation (EXECUTION of multiple insertion + dealing with Exception):

```
Station Insertion:
Input Name: Central
Input Street Name: King Fahd
Input Postal Code: 11564
Input Type: Metro
Input Working Hours: 6 AM - 11 PM
Record inserted successfully.
Insert another record? (Y/N): Y
```

```
Station Insertion:
Input Name: PNU
Input Street Name: PNU
Input Postal Code: 12345
Input Type: bus
Input Working Hours: 5 AM - 6 PM
Record inserted successfully.
Insert another record? (Y/N):
```

```
Station Insertion:
Input Name: KSU
Input Street Name: KSU
Input Postal Code: 123456
Input Type: bus
Input Working Hours: 6 AM - 5 PM
[ WARN] (main) Error: 1406-22001: Data too long for column 'postal_code' at row 1Insertion error: (conn=20) Data too long for column 'postal_code' a
Insert another record? (Y/N):
```

```
Station Insertion:
Input Name: Central
Input Street Name: King Faisal
Input Postal Code: 11565
Input Type: Bus
Input Working Hours: 7 AM - 10 PM
[ WARN] (main) Error: 1062-23000: Duplicate entry 'Central' for key 'PRIMARY'
Insertion error: (conn=11) Duplicate entry 'Central' for key 'PRIMARY'
Insert another record? (Y/N): N
```

```
Station Insertion:
Input Name: KSU
Input Street Name: KSU
Input Postal Code: a
Error: Postal code must contain only digits.
Input Postal Code: 12A
Error: Postal code must contain only digits.
Input Postal Code: 12
Error: Postal code must be exactly 5 digits.
Input Postal Code: 12321
Input Type: car
Error: Station type must be either 'Metro' or 'Bus'.
Input Type: bus
Input Working Hours (e.g., 6 AM - 11 PM): 13 AM - 5 PM
Error: Working hours must be between 1 and 12.
Input Working Hours (e.g., 6 AM - 11 PM): 5 AM - 0 PM
Error: Working hours must be between 1 and 12.
Input Working Hours (e.g., 6 AM - 11 PM): 5
Error: Working hours must contain '-' between start and end time.
Input Working Hours (e.g., 6 AM - 11 PM): 5 AM - 5 PM
Record inserted successfully.
Insert another record? (Y/N):
```

```
Station Insertion:
Input Name: AlNakheel
Input Street Name: AlNakheel, Riyadh, Kingdom of Saudi Arabia
Input Postal Code: 43212
Input Type: bus
Input Working Hours (e.g., 6 AM - 11 PM): 6 AM - 7 PM
[ WARN] (main) Error: 1406-22001: Data too long for column 'street_name' at row 1
Insertion error: (conn=42) Data too long for column 'street_name' at row 1
Insert another record? (Y/N):
```

```
Station Insertion:
Input Name: STC
Input Street Name: Alulaya
Input Postal Code: 32123
Input Type: car
Error: Station type must be either 'Metro' or 'Bus'.
Input Type: metr
Error: Station type must be either 'Metro' or 'Bus'.
Input Type: metro
Input Working Hours (e.g., 6 AM - 11 PM): 7 AM - 10 PM
Record inserted successfully.
Insert another record? (Y/N):
```

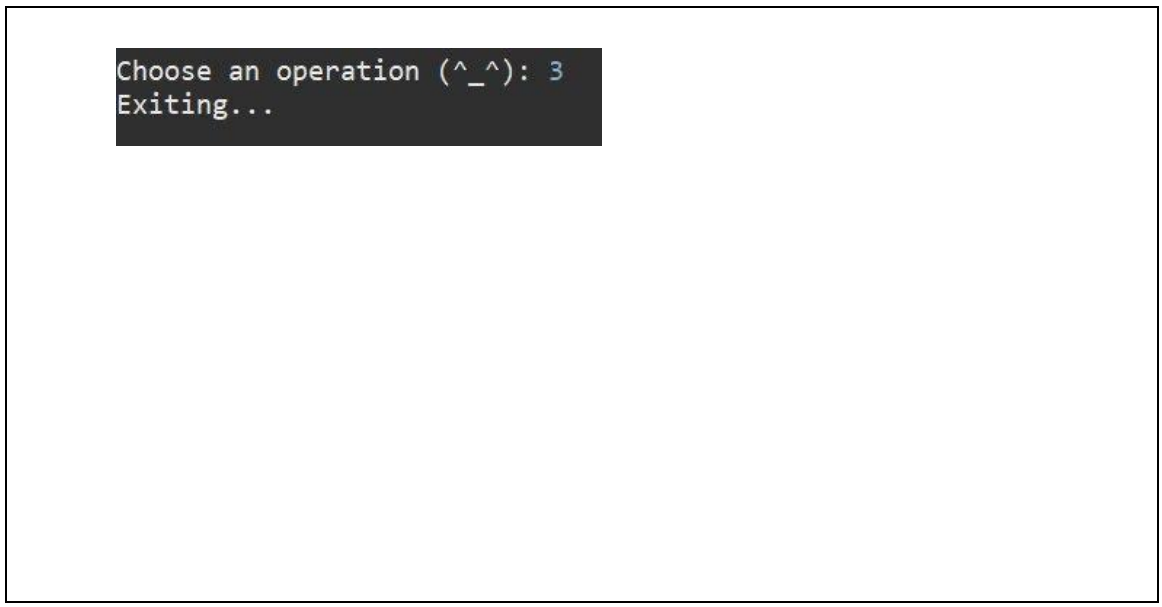
```
Station Insertion:
Input Name:
Error: Station name cannot be empty (Primary Key).
```

3) Display Operation:

```
Choose an operation (^_^): 2

--- Station Records ---
Central King Fahd       11564   Metro   6 AM - 11 PM
KSU     KSU     12321   bus     5 AM - 5 PM
PNU     PNU     12345   bus     5 AM - 6 PM
STC     Alulaya 32123   metro   7 AM - 10 PM
```

4) Exit operation:



Part 2:

Source code:

1) INSERTION code:

```
while (true) {
                    System.out.println("\nStation Insertion:");
                    String postalCode, type, workingHours;

                    System.out.print("Input Name: ");
                    String name = input.nextLine();
                    if (name.trim().equals("")) {
                       System.out.println("Error: Station name cannot be
empty (Primary Key).");
                       continue;
                    }

                    System.out.print("Input Street Name: ");
                    String streetName = input.nextLine();
                    while(true) {

                    System.out.print("Input Postal Code: ");
                    postalCode = input.nextLine();
                    try {
                       Integer.parseInt(postalCode);
```

```
                    } catch (NumberFormatException e) {
                        System.out.println("Error: Postal code must contain only digits.");
                    continue;
                    }
                    if (postalCode.length() != 5) {
                        System.out.println("Error: Postal code must be exactly 5 digits.");
                    continue;
                    }

                   break;


                    }
                    while(true) {
                    System.out.print("Input Type: ");
                    type = input.nextLine();
                    if (!type.equalsIgnoreCase("Metro") && !type.equalsIgnoreCase("Bus")) {
                        System.out.println("Error: Station type must be either 'Metro' or 'Bus'.");
                        continue;
                    }
                    else break;}

                    while(true) {

                    System.out.print("Input Working Hours (e.g., 6 AM - 11 PM): ");
                    workingHours = input.nextLine();

                    if (!workingHours.contains("-")) {
                        System.out.println("Error: Working hours must contain '-' between start and end time.");
                        continue;
                    }

                    try {
                        int dashIndex = workingHours.indexOf("-");
```

```
                    String partBeforeDash = workingHours.substring(0, dashIndex);
                    String partAfterDash = workingHours.substring(dashIndex + 2);

                     int startHour = Integer.parseInt("" + partBeforeDash.charAt(0));
                    if (Character.isDigit(partBeforeDash.charAt(1))) {
                       startHour = Integer.parseInt("" + partBeforeDash.charAt(0) + partBeforeDash.charAt(1));
                    }

                    int endHour = Integer.parseInt("" + partAfterDash.charAt(0));
                    if (Character.isDigit(partAfterDash.charAt(1))) {
                       endHour = Integer.parseInt("" + partAfterDash.charAt(0) + partAfterDash.charAt(1));
                    }

                    if (startHour < 1 || startHour > 12 || endHour < 1 || endHour > 12) {
                       System.out.println("Error: Working hours must be between 1 and 12.");
                       continue;
                    }

                 } catch (Exception e) {
                    System.out.println("Error: Invalid working hours format.");
                    continue;
                 }
                 break;
                 }


                 String sql = "INSERT INTO Station (Name, Street_Name, Postal_Code, Type, Working_Hours) " +
                              "VALUES ('" + name + "','" + streetName + "','" + postalCode + "','" + type + "','" + workingHours + "')";

                 try {
                    stmt.executeUpdate(sql);
```

```
                    System.out.println("Record inserted successfully.");
                } catch (SQLException e) {
                    System.out.println("Insertion error: " +
e.getMessage());
                }

                System.out.print("Insert another record? (Y/N): ");
                String again = input.nextLine();
                if (!again.equalsIgnoreCase("Y"))
                break;
            }
              break;
```

2) Display code:

```
try {
                ResultSet rs = stmt.executeQuery("SELECT * FROM
Station");

                System.out.println("\n--- Station Records ---");
                while (rs.next()) {
                   System.out.println(
                      rs.getString("Name") + "\t" +
                      rs.getString("Street_Name") + "\t" +
                      rs.getString("Postal_Code") + "\t" +
                      rs.getString("Type") + "\t" +
                      rs.getString("Working_Hours"));
                }
              } catch (SQLException e) {
                System.out.println("Error displaying records: " +
e.getMessage());
              }
              break;
```

3) Exit code:

```
System.out.println("Exiting...");
              input.close();
              stmt.close();
```

```
                conn.close();
                return;
```

4) Code dealing with exceptions:

1) Database connection error:

```
catch (SQLException e) {
    System.out.println("Database connection failed: " + e.getMessage());
}
```

2) Empty primary key (Station name):

```
if (name.trim().equals("")) {
    System.out.println("Error: Station name cannot be empty (Primary Key).");
    continue;
}
```

3) Postal code is not numeric:

```
try {
    Integer.parseInt(postalCode);
} catch (NumberFormatException e) {
    System.out.println("Error: Postal code must contain only digits.");
    continue;
}
```

4) Postal code length is not 5 digits:

```
if (postalCode.length() != 5) {
    System.out.println("Error: Postal code must be exactly 5 digits.");
    continue;
}
```

5) Invalid station type (not Metro or Bus):

```
if (!type.equalsIgnoreCase("Metro") && !type.equalsIgnoreCase("Bus")) {
    System.out.println("Error: Station type must be either 'Metro' or 'Bus'.");
    continue;
```

```
        }

        6) Invalid working hours format (missing '-'):

        if (!workingHours.contains("-")) {
            System.out.println("Error: Working hours must contain '-' between
start and end time.");
            continue;
        }

        7) Working hours not between 1 and 12:

        if (startHour < 1 || startHour > 12 || endHour < 1 || endHour > 12) {
            System.out.println("Error: Working hours must be between 1 and
12.");
            continue;
        }

        8) Insert operation SQL error:

        try {
            stmt.executeUpdate(sql);
            System.out.println("Record inserted successfully.");
        } catch (SQLException e) {
            System.out.println("Insertion error: " + e.getMessage());
        }

        9) Display records SQL error:

        try {
            ResultSet rs = stmt.executeQuery("SELECT * FROM Station");
        } catch (SQLException e) {
            System.out.println("Error displaying records: " + e.getMessage());
        }




        5) The Whole Source code:
        import java.sql.*;
        import java.util.*;
        public class MARIA {
```

```
  public static void main(String[] args) {
Connection conn = null;
Scanner input = new Scanner(System.in);

String url = "jdbc:mariadb://localhost:3306/project";
String user = "root";
String pwd = "sqlpassword";

try {
   conn = DriverManager.getConnection(url, user, pwd);
   Statement stmt = conn.createStatement();
   System.out.println("Connected to the database successfully.");

   while (true) {
      System.out.println("\n==== Station Table Menu ====");
      System.out.println("1. Insert new record.");
      System.out.println("2. Display all records.");
      System.out.println("3. Exit.");
      System.out.print("Choose an operation (^_^): ");
      String choice = input.nextLine();

      switch (choice) {
         case "1":
          while (true) {
             System.out.println("\nStation Insertion:");
             String postalCode, type, workingHours;

             System.out.print("Input Name: ");
             String name = input.nextLine();
             if (name.trim().equals("")) {
                System.out.println("Error: Station name cannot be
empty (Primary Key).");
                continue;
             }

             System.out.print("Input Street Name: ");
             String streetName = input.nextLine();
             while(true) {

             System.out.print("Input Postal Code: ");
             postalCode = input.nextLine();
             try {
```

```
                    Integer.parseInt(postalCode);
                } catch (NumberFormatException e) {
                    System.out.println("Error: Postal code must contain only digits.");
                continue;
                }
                if (postalCode.length() != 5) {
                    System.out.println("Error: Postal code must be exactly 5 digits.");
                continue;
                }

               break;


                }
                while(true) {
                System.out.print("Input Type: ");
                type = input.nextLine();
                if (!type.equalsIgnoreCase("Metro") && !type.equalsIgnoreCase("Bus")) {
                    System.out.println("Error: Station type must be either 'Metro' or 'Bus'.");
                    continue;
                }
                else break;}

                while(true) {

                System.out.print("Input Working Hours (e.g., 6 AM - 11 PM): ");
                workingHours = input.nextLine();

                if (!workingHours.contains("-")) {
                    System.out.println("Error: Working hours must contain '-' between start and end time.");
                    continue;
                }

                try {
                    int dashIndex = workingHours.indexOf("-");
```

```
                    String partBeforeDash = workingHours.substring(0, dashIndex);
                    String partAfterDash = workingHours.substring(dashIndex + 2);

                     int startHour = Integer.parseInt("" + partBeforeDash.charAt(0));
                    if (Character.isDigit(partBeforeDash.charAt(1))) {
                       startHour = Integer.parseInt("" + partBeforeDash.charAt(0) + partBeforeDash.charAt(1));
                    }

                    int endHour = Integer.parseInt("" + partAfterDash.charAt(0));
                    if (Character.isDigit(partAfterDash.charAt(1))) {
                       endHour = Integer.parseInt("" + partAfterDash.charAt(0) + partAfterDash.charAt(1));
                    }

                    if (startHour < 1 || startHour > 12 || endHour < 1 || endHour > 12) {
                       System.out.println("Error: Working hours must be between 1 and 12.");
                       continue;
                    }

                 } catch (Exception e) {
                    System.out.println("Error: Invalid working hours format.");
                    continue;
                 }
                 break;
                 }


                 String sql = "INSERT INTO Station (Name, Street_Name, Postal_Code, Type, Working_Hours) " +
                              "VALUES ('" + name + "','" + streetName + "','" + postalCode + "','" + type + "','" + workingHours + "')";

                 try {
                    stmt.executeUpdate(sql);
```

```
                    System.out.println("Record inserted successfully.");
                } catch (SQLException e) {
                    System.out.println("Insertion error: " +
e.getMessage());
                }

                System.out.print("Insert another record? (Y/N): ");
                String again = input.nextLine();
                if (!again.equalsIgnoreCase("Y"))
                break;
            }
              break;

            case "2":
              try {
                ResultSet rs = stmt.executeQuery("SELECT * FROM
Station");

                System.out.println("\n--- Station Records ---");
                while (rs.next()) {
                   System.out.println(
                     rs.getString("Name") + "\t" +
                     rs.getString("Street_Name") + "\t" +
                     rs.getString("Postal_Code") + "\t" +
                     rs.getString("Type") + "\t" +
                     rs.getString("Working_Hours"));
                }
              } catch (SQLException e) {
                System.out.println("Error displaying records: " +
e.getMessage());
              }
              break;

            case "3":
              System.out.println("Exiting...");
              input.close();
              stmt.close();
              conn.close();
              return;

            default:
              System.out.println("Invalid choice. Please try again.");
```

```
                }
            }

        } catch (SQLException e) {
            System.out.println("Database connection failed: " +
e.getMessage());
        }
    }
        }
```